

Kubernetes (K8s)

Java Backend Interview — Short Notes

1. K8s Architecture

1.1 Control Plane (Master Node)

Component	Role	Notes
kube-apiserver	REST API front-end — all kubectl commands and components communicate through it	Validates requests, auth/authz, persists to etcd
etcd	Distributed key-value store — single source of truth for entire cluster state	All K8s objects stored here. Back up etcd = back up cluster
kube-scheduler	Assigns pods to nodes — considers resources, constraints, affinity	Finds best node per pod's requirements. Does not run pods
kube-controller-manager	Runs controllers — ReplicaSet, Deployment, Node, Job, etc.	Watch-loop: observe state, compare to desired, reconcile
cloud-controller-manager	Cloud-provider specific controllers — LB provisioning, node management	AWS/GCP/Azure specific logic separated from core K8s

1.2 Worker Node

Component	Role	Notes
kubelet	Node agent — ensures containers in pods are running and healthy	Communicates with container runtime. Executes pod specs
kube-proxy	Network proxy — maintains iptables/ipvs rules for Service routing	Implements Services — routes traffic to correct pods
Container Runtime	Runs containers — containerd (default), CRI-O, Docker (deprecated)	CRI (Container Runtime Interface) abstraction
Pod	Smallest deployable unit — one or more containers sharing network/storage	Ephemeral — killed and recreated, not restarted in place

2. Core Kubernetes Objects

Object	Purpose	Key Fields	Relations
Pod	Runs containers. Ephemeral. Has IP within cluster	spec.containers, resources, volumes, nodeSelector	Created by Deployment/StatefulSet/DaemonSet
ReplicaSet	Ensures N pod replicas running. Self-healing	spec.replicas, selector, template	Managed by Deployment. Rarely created directly
Deployment	Manages ReplicaSets. Rolling updates, rollbacks	spec.replicas, strategy, template, selector	Creates/manages ReplicaSets
StatefulSet	Stateful apps — stable identity, ordered deployment	serviceName, volumeClaimTemplates, updateStrategy	Each pod gets stable hostname: pod-0, pod-1
DaemonSet	Run one pod per node — monitoring agents, log collectors	selector, template, updateStrategy	Automatically adds pod to new nodes

Job	Run to completion — batch tasks. Retry on failure	completions, parallelism, backoffLimit	Creates pods, tracks completion
CronJob	Scheduled Job — cron syntax	schedule (cron), jobTemplate, concurrencyPolicy	Creates Jobs on schedule
Service	Stable network endpoint for pods. Load balances	selector, ports, type (ClusterIP/NodePort/LB)	Routes to pods matching selector
Ingress	HTTP/HTTPS routing — path/host-based, TLS	rules (host, paths), tls, ingressClassName	Requires IngressController (nginx, traefik)
ConfigMap	Non-sensitive config — env vars or mounted files	data (key-value pairs)	Referenced by pods as env or volume
Secret	Sensitive data — base64 encoded, encrypted at rest	data (base64), type (Opaque, kubernetes.io/tls)	Referenced by pods. Use ExternalSecrets for Vault
PersistentVolume	Cluster-level storage resource provisioned by admin	capacity, accessModes, storageClassName, volumeMode	Bound to PVC by StorageClass
PersistentVolumeClaim	Pod's request for storage — size, access mode	resources.requests.storage, accessModes, storageClass	Pod mounts PVC as volume
StorageClass	Dynamic volume provisioning — defines provisioner	provisioner, parameters, reclaimPolicy	Creates PV on-demand when PVC created
HPA	Horizontal Pod Autoscaler — scale pods on metrics	scaleTargetRef, minReplicas, maxReplicas, metrics	Scales Deployment/StatefulSet replicas
RBAC	Role-based access control — ServiceAccount, Role, RoleBinding	rules (verbs, resources), subjects	ServiceAccount → RoleBinding → Role
Namespace	Virtual cluster within cluster — isolation, quota	metadata.name	Most objects are namespace-scoped

3. Pod Specification Deep Dive

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  namespace: production
  labels:
    app: order-service
version: v1.2.3
spec:
  replicas: 3
  selector:
    matchLabels:
      app: order-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1 # extra pods during update
      maxUnavailable: 0 # zero-downtime — no pods removed until new ready
  template:
    metadata:
      labels:
        app: order-service

```

```
spec:
  serviceAccountName: order-sa
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
  containers:
    - name: order-service
  image: myregistry/order-service:1.2.3
  ports:
    - containerPort: 8080
  env:
    - name: SPRING_PROFILES_ACTIVE
      value: prod
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: order-db-secret
          key: password
  resources:
    requests:
      memory: "256Mi" # K8s uses this for scheduling
      cpu: "250m" # 250 millicores = 0.25 CPU
    limits:
      memory: "512Mi" # OOMKill if exceeded
      cpu: "500m" # throttled if exceeded
  livenessProbe:
    httpGet: { path: /actuator/health/liveness, port: 8080 }
    initialDelaySeconds: 30
    periodSeconds: 10
    failureThreshold: 3
  readinessProbe:
    httpGet: { path: /actuator/health/readiness, port: 8080 }
    initialDelaySeconds: 20
    periodSeconds: 5
  volumeMounts:
    - name: config-volume
      mountPath: /config
  volumes:
    - name: config-volume
  configMap:
    name: order-service-config
```

4. Services & Networking

4.1 Service Types

Type	Access	Description	Use Case
ClusterIP	Internal only	Virtual IP within cluster. DNS: svc.namespace.svc.cluster.local	Internal service-to-service. Default type
NodePort	External via node IP:port	Exposes port 30000-32767 on every node. Not for production	Dev/test external access

LoadBalancer	External via cloud LB	Provisions cloud load balancer (AWS ELB, GCP LB). Expensive — one LB per service	Production external services (prefer Ingress)
ExternalName	DNS CNAME alias	Maps service to external DNS name — no proxy	Migrate external services into cluster namespace
Headless	Direct pod IPs	clusterIP: None. DNS returns pod IPs. StatefulSets get stable DNS per pod	StatefulSets, service meshes, direct pod connection

4.2 Service & Ingress YAML

```
# ClusterIP Service
apiVersion: v1
kind: Service
metadata: { name: order-service, namespace: production }
spec:
  selector: { app: order-service }
  ports:
    - port: 80 # service port
  targetPort: 8080 # container port
  type: ClusterIP

# Ingress – path-based routing
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  tls:
    - hosts: [api.example.com]
  secretName: api-tls-secret
  rules:
    - host: api.example.com
    http:
      paths:
        - path: /orders
        pathType: Prefix
        backend:
          service: { name: order-service, port: { number: 80 } }
        - path: /users
        pathType: Prefix
        backend:
          service: { name: user-service, port: { number: 80 } }
```

5. Scaling & Deployment Strategies

5.1 Horizontal Pod Autoscaler (HPA)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: order-service-hpa }
spec:
```

```

scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: order-service
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
  resource:
    name: cpu
    target: { type: Utilization, averageUtilization: 70 }
    - type: External # custom metric – Kafka consumer lag
  external:
    metric: { name: kafka_consumer_lag }
    target: { type: AverageValue, averageValue: "1000" }

```

5.2 Deployment Strategies

Strategy	How it Works	Downtime	Risk	Use Case
RollingUpdate	Replace pods gradually — maxSurge new pods before removing old	Zero	Both versions live briefly	Default — zero-downtime standard deployments
Recreate	Kill all old pods, then create all new	Yes — gap between old down and new up	No mixed versions	When v1 and v2 cannot coexist (breaking schema)
Blue-Green	Two full environments. Switch traffic instantly at LB/Service	Zero	Double infrastructure cost	Instant rollback capability, large releases
Canary	Route small % (e.g. 5%) to new version. Gradually increase	Zero	Small % exposed to new version	High-risk changes, test with real traffic first

```

# Canary via labels + service (simple approach)
# v1: 9 pods labeled app=api,version=v1
# v2: 1 pod labeled app=api,version=v2
# Service selector: app=api (selects both) → ~10% v2 traffic

```

6. Health Probes

Probe	On Failure	Checks	K8s Action
Liveness	Container is dead/stuck	Is app alive? Can it serve any request at all?	Restart container
Readiness	Container not ready for traffic	Is app ready to serve requests? DB connected? Warm cache?	Remove from Service endpoints — no traffic routed
Startup	App didn't start in time	Has app finished starting? Protects slow-starting apps	Liveness/readiness not checked until startup succeeds

• Probe methods:

- httpGet: HTTP GET — 2xx/3xx = healthy. /actuator/health is standard for Spring Boot
- tcpSocket: check TCP connection to port — lower overhead
- exec: run command inside container — exit code 0 = healthy

■ grpc: gRPC health check protocol

✓ Spring Boot: `management.endpoint.health.probes.enabled=true` exposes `/health/liveness` and `/health/readiness`

7. ConfigMaps & Secrets

```
# ConfigMap – inject as env vars
apiVersion: v1
kind: ConfigMap
metadata: { name: app-config }
data:
  LOG_LEVEL: INFO
  CACHE_TTL: "300"
  application.yaml: |
  server.port: 8080
  spring.profiles.active: prod
# In pod – inject all ConfigMap keys as env vars
envFrom:
  - configMapRef: { name: app-config }
# Or mount as file
volumes:
  - name: config
  configMap: { name: app-config }
volumeMounts:
  - name: config
  mountPath: /config
# Secret
apiVersion: v1
kind: Secret
metadata: { name: db-secret }
type: Opaque
stringData: # K8s base64-encodes automatically
  DB_PASSWORD: supersecret
  DB_URL: jdbc:postgresql://prod-db:5432/orders
```

■ K8s Secrets base64-encoded, not encrypted by default. Enable etcd encryption at rest. Use ExternalSecrets Operator with Vault/AWS Secrets Manager for production

8. RBAC — Access Control

```
# ServiceAccount – pod identity
apiVersion: v1
kind: ServiceAccount
metadata: { name: order-sa, namespace: production }
# Role – namespace-scoped permissions
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: order-role, namespace: production }
rules:
  - apiGroups: ["" ]
    resources: [pods, configmaps]
    verbs: [get, list, watch]
# RoleBinding – bind ServiceAccount to Role
apiVersion: rbac.authorization.k8s.io/v1
```

```

kind: RoleBinding
metadata: { name: order-rolebinding, namespace: production }
subjects:
- kind: ServiceAccount
  name: order-sa
  namespace: production
roleRef:
  kind: Role
  name: order-role
  apiGroup: rbac.authorization.k8s.io

```

- **ClusterRole / ClusterRoleBinding** — cluster-scoped (all namespaces). For cluster-wide resources like nodes, PVs

9. Essential kubectl Commands

Command	Description
kubectl get pods -n ns -o wide	List pods in namespace with node and IP. -A for all namespaces
kubectl describe pod pod-name	Full details — events, conditions, probe status, resource limits. Debug pod issues
kubectl logs pod-name -c container -f	Stream logs. -c for specific container in multi-container pod. --previous for crashed container
kubectl exec -it pod -- bash	Interactive shell in pod. Use sh if no bash. -- separates kubectl and container args
kubectl apply -f manifest.yaml	Create or update resources declaratively. Preferred over create for idempotency
kubectl delete -f manifest.yaml	Delete resources defined in file. Or kubectl delete deploy/name
kubectl rollout status deploy/name	Wait for rollout to complete. Watch rolling update progress
kubectl rollout undo deploy/name	Roll back to previous ReplicaSet version. --to-revision=N for specific
kubectl rollout history deploy/name	Show revision history. Add --revision=N for details
kubectl scale deploy/name --replicas=5	Scale deployment to 5 replicas immediately
kubectl top pods / kubectl top nodes	Resource usage (CPU/memory). Requires metrics-server
kubectl port-forward pod/name 8080:8080	Forward local port to pod. Debug without exposing service
kubectl cp pod:/remote/path ./local	Copy files to/from pod. Debug, extract logs/artifacts
kubectl get events --sort-by=.lastTimestamp	View cluster events — debugging pod scheduling/errors
kubectl explain deploy.spec	API docs inline — explore object spec fields without googling
kubectl label pod name key=value	Add label to resource. Useful for manual canary testing
kubectl taint node name key=value:NoSchedule	Prevent pods from scheduling on node unless they tolerate taint
kubectl drain node --ignore-daemonsets	Evict all pods from node for maintenance. Cordon + evict
kubectl cordon/uncordon node	Mark node unschedulable / schedulable. No eviction of existing pods
kubectl diff -f manifest.yaml	Show diff between current and desired state — safe preview before apply

10. Interview Questions

Architecture

Q1. What is Kubernetes? Why use it?

→ Container orchestration platform. Automates deployment, scaling, self-healing, service discovery, load balancing, rolling updates, secret management. Solves: how do I run N containers reliably across many machines?

Q2. What are control plane components?

→ kube-apiserver (REST API gateway), etcd (cluster state store), kube-scheduler (assign pods to nodes), kube-controller-manager (reconciliation controllers), cloud-controller-manager (cloud provider integration)

Q3. What does etcd store?

→ ALL cluster state — every object (pods, deployments, services, configmaps, secrets). If etcd is lost, cluster state is lost. Always back it up. 3 or 5 etcd nodes for HA (odd number for quorum)

Q4. What is a kubelet?

→ Agent running on each worker node. Watches API server for pod assignments. Ensures containers in pods are running per spec. Reports node/pod health back to control plane. Communicates with container runtime

Q5. What is kube-proxy?

→ Runs on each node. Maintains iptables/IPVS rules to implement Services. Routes traffic destined for a Service's ClusterIP to one of the backing pod IPs. Handles load balancing at network level

Objects

Q6. What is a Pod? Why is it the basic unit instead of container?

→ Pod: one or more tightly coupled containers sharing network namespace (same IP/ports) and volumes. Groups related containers (app + sidecar). Containers in pod communicate via localhost

Q7. Deployment vs StatefulSet vs DaemonSet?

→ Deployment: stateless apps — pods interchangeable, random names. StatefulSet: stateful apps — stable hostname (pod-0, pod-1), ordered deploy/scale, persistent storage per pod. DaemonSet: exactly one pod per node — monitoring agents, log collectors

Q8. What is a Service? Why needed?

→ Pods are ephemeral — IP changes on restart. Service provides stable ClusterIP + DNS name + load balancing across pods. kube-proxy routes traffic. Four types: ClusterIP (internal), NodePort (node port), LoadBalancer (cloud LB), ExternalName (DNS alias)

Q9. ClusterIP vs NodePort vs LoadBalancer?

→ ClusterIP: cluster-internal only, default. NodePort: exposed on every node's IP at fixed port (30000-32767), accessible externally — not for production. LoadBalancer: provisions cloud LB, production external access. Prefer Ingress + ClusterIP for HTTP traffic

Q10. What is Ingress? How does it differ from LoadBalancer Service?

→ Ingress: HTTP routing rules — path/host based, TLS termination, one LB for many services. Requires IngressController (nginx/traefik). LoadBalancer Service: one cloud LB per service — expensive. Ingress = one LB for all HTTP services via routing rules

Operations

Q11. Difference between liveness and readiness probes?

→ Liveness: is container alive? Fail → restart. Use for deadlock detection. Readiness: is container ready for traffic? Fail → remove from Service endpoints. Use for warmup, dependency checks. Both failing → restart but no traffic to restarting pod

Q12. What happens during a rolling update?

→ K8s creates new pods (maxSurge), waits for them to become ready, then removes old pods (maxUnavailable). maxSurge=1, maxUnavailable=0 = zero-downtime. New and old versions run simultaneously during update

Q13. How does HPA work?

→ HPA controller queries metrics-server every 15s. Computes desired replicas: $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} * (\text{currentMetric}/\text{targetMetric}))$. Scales Deployment. Needs requests set on pods for CPU-based scaling

Q14. What are resource requests vs limits?

→ Requests: guaranteed allocation — scheduler uses for placement. Pod only scheduled on node with enough available. Limits: maximum allowed. CPU: throttled if exceeded. Memory: OOMKilled (container killed) if exceeded. Always set both

Q15. What is ConfigMap vs Secret?

→ ConfigMap: non-sensitive config — environment, config files. Secret: sensitive — passwords, tokens, certs. Both mounted as env vars or files. Secret values base64-encoded (not encrypted by default). Encrypt etcd at rest + ExternalSecrets for production

Advanced

Q16. What is a Namespace? What are its uses?

→ Virtual cluster partitioning. Use for: environment isolation (dev/staging/prod), team isolation, resource quotas per namespace, RBAC boundaries. Cluster-scoped resources (Nodes, PVs, ClusterRoles) are not namespaced

Q17. What is RBAC in K8s?

→ Role-based access control: ServiceAccount (pod identity) → RoleBinding → Role (permissions). Role: namespace-scoped. ClusterRole: cluster-scoped. Principle of least privilege — each pod should have minimum required permissions

Q18. What are taints and tolerations?

→ Taint on node repels pods (NoSchedule, PreferNoSchedule, NoExecute). Toleration on pod allows it to be scheduled on tainted node. Use: dedicate GPU nodes for ML pods, keep infra pods on master, spot instance taint

Q19. What is node affinity?

→ Rules for which nodes pods can/prefer to schedule on. requiredDuringScheduling: hard constraint (like NodeSelector but expressive). preferredDuringScheduling: soft preference. podAffinity/podAntiAffinity: relative to other pods

Q20. What happens when a node goes down?

→ Node controller marks node NotReady after timeout (~40s). After pod-eviction-timeout (~5m), pods evicted and rescheduled on healthy nodes. StatefulSet pods not automatically rescheduled — require manual intervention or PodDisruptionBudget

Q21. What is a PodDisruptionBudget (PDB)?

→ Limits voluntary disruption — min pods available during node drain or rolling update. minAvailable: 2 means at least 2 pods must be up. maxUnavailable: 1 means at most 1 pod down. Ensures high availability during maintenance

Q22. How do you debug a pod stuck in Pending?

→ kubectl describe pod: check Events section. Common reasons: insufficient resources (CPU/memory), no nodes match nodeSelector/affinity, taints not tolerated, PVC not bound, image pull failure. Check kubectl get nodes -o wide for node capacity

Q23. What is a sidecar container?

→ Additional container in same pod sharing network and storage. Common: log shipper (Fluentd), service mesh proxy (Envoy/Istio), secret injector (Vault agent). Main container focuses on business logic; sidecar handles cross-cutting concerns

Q24. What is Helm?

→ Package manager for K8s — templates + values → manifests. Chart: collection of K8s YAML templates. Values: customizable per environment. helm install/upgrade/rollback. Simplifies complex multi-resource deployments

Q25. What is the difference between kubectl apply and kubectl create?

→ create: imperative — fails if resource exists. apply: declarative — create or update. Computes diff from last-applied-configuration annotation. Use apply for GitOps and idempotent CI/CD pipelines